

Unified Robotics III Final Lab

S. Dhingra, B. Lavoie, M. Sacca, F. Schroder, *Team 1*

ABSTRACT

The purpose of this lab was to design and implement an object-oriented approach for identifying, locating, and collection of differently colored objects on a checkerboard base, utilizing a 5-Degree of Freedom (4+1 Gripper) Dynamixel OpenManipulator-X Robotic Arm and USB Webcam. Using MATLAB, we designed a state machine which iteratively searches for the objects and determines their color and location via a multi-layered image processing pipeline, and then utilizes inverse/forward kinematics in conjunction with coordinate transforms to effectively pick up and sort out differently colored balls into bins of predetermined location.

I. INTRODUCTION

A. Background

The goal of this lab was to use what we learned from the previous labs to pick up and sort different colored balls from a checkered grid with our robotic arm. To do this, we utilized forward, inverse, and differential kinematics, trajectory generation, and finally computer vision.

To begin, we connected the camera to our ubuntu system and used the camera calibration app in MATLAB to calibrate it. We then proceeded to write a function that used geometry to convert a vector of 2d points on the checkerboard into a 3d point in the robot reference frame.

For detecting the balls, we used a color mask from the Color Thresholder app and used the binary image to detect the center of the ball. Our computer vision pipeline was then completed with a geometrical way to calculate the height of the balls.

We then created a function to pick up, and sort balls given the location of the balls and implemented a state machine to constantly search and sort the balls as they are placed on the board.

B. Motivation

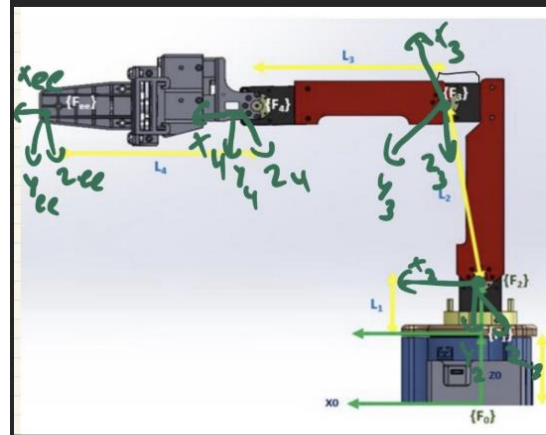
This lab was created so that we can use what we have learned in all our lectures and previous labs. In previous labs, we implemented forward, and inverse kinematics on our robotic arm. We then created trajectory generation, and differential kinematics. The motivation behind this final lab was to learn, implement, and perfect all these topics, as well as computer vision.

II. METHODS

A. Forward and Inverse Kinematic Derivations

In previous labs we used the geometry of the robot to write forward and inverse kinematics that could position the end effector at and point in space that we wanted it to.

To begin this process, we created a DH table for reference frame transitions between joints on the robot. This DH table was made through geometric analysis and precise measurements. After implementing this table into MATLAB, we experimented and verified our results before proceeding onward.



Link	Theta	D	A	Alpha
0	0	60	0	0
1	0	36.3	0	-90
2	Theta 2 star - 79.4	0	130	0
3	Theta 3 star + 79.4	0	124	0
4	Theta 4 star	0	133.4	0

Figure 1: Geometrically Derived DH Table Sketch

The Forward Kinematics were now complete, as we could determine the position and orientation of our end effector given a set of joint angles. The next step of the FK/ IK process was to derive the Inverse Kinematics using geometry.

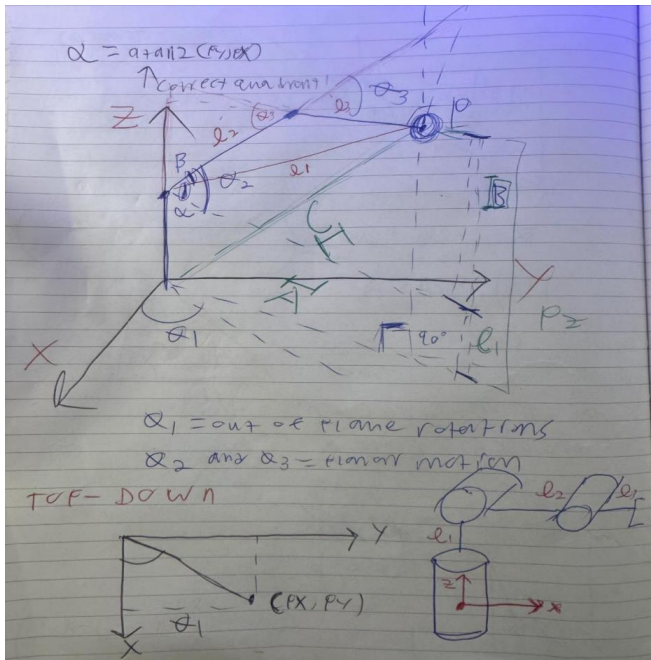


Figure 2: Sketch Used to Derive Inverse Kinematics (Using P_x , P_y , and P_z)

Using this sketch, we were able to set up a system of equations for P_x , P_y , and P_z . Using our set of three variables and three unknowns (θ_1 , θ_2 , and θ_3), we derived an equation for each of them so they could be described in terms of robot geometry and the end effector position. It is important to note that while we have four working joints on this robot, and therefore four θ angles, it is not possible to cleanly solve the IK of a robot arm with more degrees of freedom than spatial dimensions. To work around this issue, we used a process known as kinematic decoupling. This process had us set our end effector angle to a constant position, known as *Gamma*, and solve for a set of P prime variables which dictated the actual end effector position we were controlling. We then expressed the actual end effector position in terms of the prime variable position to create a mathematical system that could properly perform inverse kinematics with four independent joint angles. Our robot arm only operates in the “elbow down” configuration, so there is no need to perform any more advanced calculations than previously described.

B. Forward and Inverse Velocity Derivations

In Lab four, we used MATLAB's Jacobian function with our previously derived DH table to convert our reference frame translations for position into a velocity algorithm for the end effector. Seen here (figure 3) is a snippet of our code.

```
lin_jacob = jacobian(fk(1:3, 4), [theta_1, theta_2, theta_3, theta_4]);
T_01 = robot.dh2fk(dh_table(1:2, :));
T_02 = robot.dh2fk(dh_table(1:3, :));
T_03 = robot.dh2fk(dh_table(1:4, :));
T_04 = robot.dh2fk(dh_table(1:5, :));
ang_jacob = [T_01(1:3, 3), T_02(1:3, 3), T_03(1:3, 3), T_04(1:3, 3)];
```

Figure 3: Use of MATLAB Functionality for Jacobian Derivations

The MATLAB Jacobian function uses the method of partial derivatives to compute a partially derived matrix with

respect to the input parameters. The top line is responsible for the linear portion of the arm's Jacobian Matrix. For the angular portion, we used the following five lines, creating independent transformation matrixes for simplification and then combining them into one full angular Jacobian matrix. This function is called through `Robot.Jacob3001`. It uses the current joint angle positions as input before being multiplied by the joint velocity vectors to produce the linear and angular velocities of the end effector to give the current conditions.

C. System Architecture

Our system is based around a state machine that integrates our computer vision and robot manipulation. The main control script, `sort_balls`, implements the state machine and coordinates all other classes. The `Robot` class handles all robotic motion and kinematics, while the `ImageProcessor` class handles all computer vision and ball detection.

At the start of execution, we initialized the robot and sent it to a home position. The home position was chosen as a position that would disallow the robot to be visible in the camera's workspace. We then defined several constants, including `z_offset`, `z_ball`, and `waitTime`, which were the deposit height, ball height, and time between cycles respectively. Variables were also created to store detected ball colors and positions.

The control logic ended up being a state machine consisting of 2 states. The first state is the Searching state. Within this state, the system attempts to detect and locate balls that are placed on the workspace. It does this using the `detect_balls()` function, which returns a list of ball positions and colors. When no balls are detected, the system briefly pauses, and restarts the search.

The second state is the picking state. Within this, the robot iterates through the given list of detected balls and then picks up the ball based off its given position and puts them into bins based off its given color. When the robot is done cycling through all balls and putting them into their correct bins, it goes back to its original home position, and switches back to the searching state. Due to the indefinite loop that this state machine runs on, the robot has continuous autonomous operation.

The provided framework takes many considerations into account. First and foremost, it allows for full modularity in the system. There are multiple things that are happening at the same time, and working together, which improves the debugging process, and makes it easier to maintain the code. One change we would make to the architecture in the future is that we would make a new class which would be made to plot data and make figures. A new class would limit the amount of repetition in the code, especially in the debug portions.

```

open_gripper()
blocking_js_move(over ball)
blocking_ts_move(down ball)
close_gripper()
blocking_js_move(over bin)
open_gripper()

```

Figure 4: *pick_up_ball* pseudocode

```

35 while true
36     if robotState == "Searching"
37         % Searches for a ball
38         % ballPose = robot.detect_balls();
39
40         [ballColors, ballPoses] = imgProcessor.detect_balls();
41
42         % If the ball Pose is not empty (ball on board)
43         if ~isempty(ballPoses)
44             robotState = "Picking"; % Go into picking
45         end
46         disp("Search Pause")
47         pause(waitTime);
48     elseif robotState == "Picking"
49
50         for i = 1:size(ballPoses, 1)
51             ballPose = ballPoses(i,:);
52             ballColor = ballColors(i);
53             robot.pick_up_ball(ballPose, ballColor, z_offset, z_ball);
54         end
55
56         ballColors = [];
57         ballPoses = [];
58
59         robotState = "Searching";
60     end
61 end

```

Figure 5: *state machine code snippet*

D. Image Processing Pipeline

Our 4-Step Image Processing Pipeline allows for incredibly accurate and extremely reliable centroid detection and locating. Prior to the first step is the initialization of the ImageProcessor class and calibration of its associated webcam. This is followed by the generation of a static mask cropping out the area surrounding the checkerboard to be used later in Step 2 of the pipeline. Once this occurs, the state machine is started, and with it the Image Processing Pipeline.



Figure 6: *Calibrated, unmasked image from webcam.*

Step 1 of this process is receiving an image taken via the calibrated webcam, which has the fisheye effect accounted for via calibrated image distortion. This allows the checkerboard pattern to appear consistent and leads to reliable coordinate transformation. MATLAB utilizes the checkerboard to calculate the intrinsic camera matrix, as the predetermined spacing (25mm per square) allows the relationship between pixels and real-world space to be confirmed.

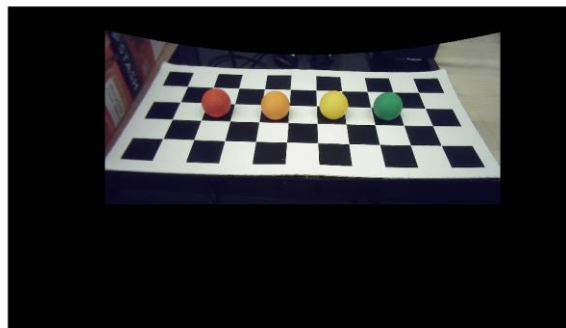


Figure 7: *Captured image following static masking.*

Following the image being received in Step 1, Step 2 utilizes the static mask generated upon calibration of the webcam by applying it to the current image, keeping only the checkerboard and the balls on it within a set margin. This prevents objects outside the predetermined area from being mistaken as targets for collection.



Figure 8: *Modified image following ball masking.*

Step 3 applies the first color mask to the image, not in a binary fashion, but in such a way that it masks the checkerboard and other the area left uncropped by the static mask completely out of the image, isolating the colored balls into their own blank frame. This is an essential step in the optimization of this process, allowing the balls and their colors to be thoroughly separated, and avoiding masses of noise in the final image, which effectively eradicates

centroid detection error.

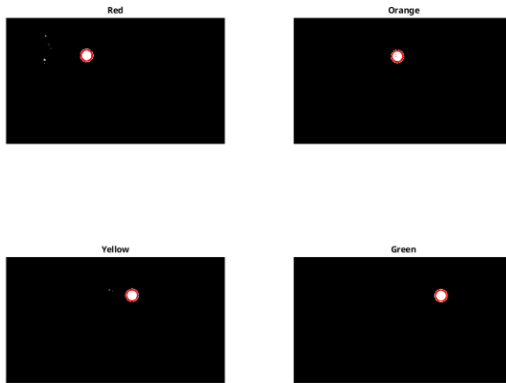


Figure 9: Final Images following individual color masking and centroid detection.

The fourth and final step of the Image Processing Pipeline addresses the individual color masking and centroid detection. Individual color masks tailored towards the shades of red, orange, yellow, and green exhibited by the balls are applied in binary fashion to the already masked image produced in the previous step. This new collection of four images is iterated through, and the MATLAB *regionprops* function is used to detect centroids within the binary images, collecting information relating to their position and size. This information is then collected into an array, with its corresponding color information being collected in a separate, identically-indexed array. This pair of vectors is then passed on to be transformed to the checkerboard frame via *pointToWorld*, then to the robot frame, and then finally corrected for exact world location.

Once the camera was able to uniquely identify each ball and assign it a position using points to the world, we encountered a common issue in image processing. The camera's projection of the placement of the ball onto the world was not actually in the correct position, as the optical view of the camera projects the position of the ball to be slightly behind the actual ball position due to the camera's inability to perceive object depth with a single 2d reference image. This problem was anticipated by the lab document, with an illustration that describes the issue shown here:

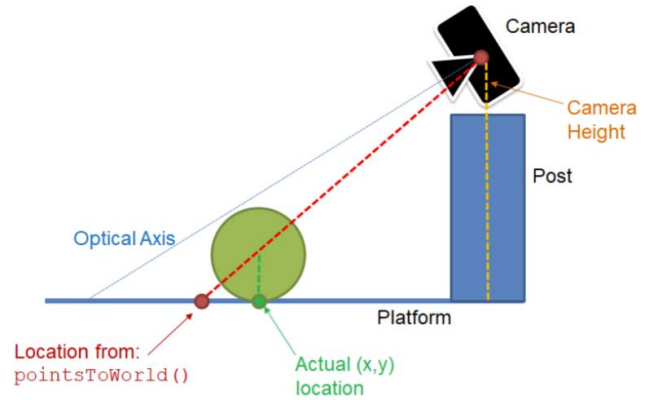


Figure 10: XY Location Error Analysis

To solve this issue, we used basic trigonometry, derived from the sketches shown below:

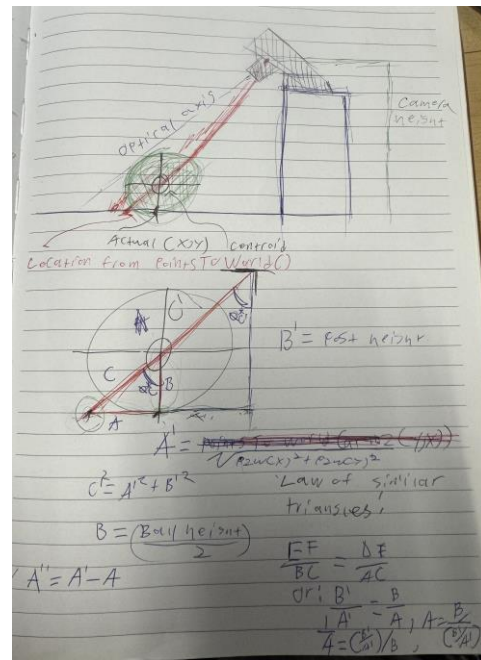


Figure 11: Trigonometry Derived Radial XY Position

This image shows how we used the law of similar triangles to solve for the radial distance from the camera to the true XY position of the ball. We are able to use this method because we know the camera height, the ball height, the base length of the triangle A Prime, and that both of the triangles pictured are similar. Knowing these things, we were then able to solve for the x and y components of our radial distance, giving us the true XY position of the ball:

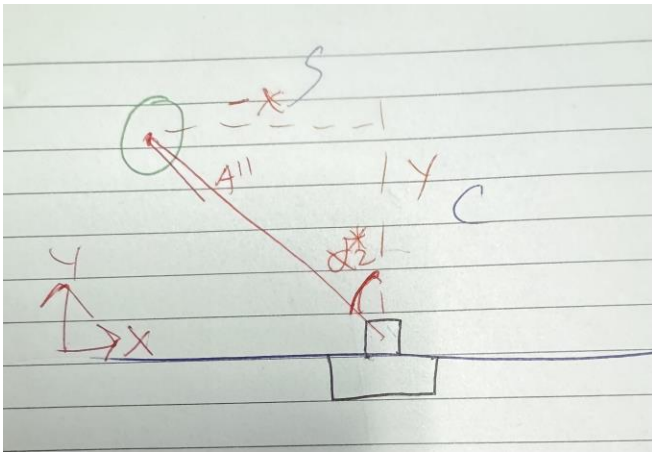


Figure 12: Derived XY Position using A Double Prime, or Radial Position

It is important to note that we initially had no frame of reference relative to the camera when beginning this process. This means that in our code implementation, we had to compensate for this by translating the camera coordinates into base frame coordinates. Theta 2, shown in this sketch, is simply the $\arctan(y,x)$ of the points to world y and x coordinates. By combining the sine and cosine components with the A double prime (or radial distance from the camera), we are able to extract the true x and y position coordinates of the ball.

III. RESULTS

A. Robot Performance

After implementation of all kinematics, computer vision, and trajectory generation, the robot was able to detect, pick up, and sort different colored balls that were placed on the grid. The camera system first detected where each ball was placed and identified the color. This was done with an image processing pipeline, which consists of multiple color masks made by the color threshold app in MATLAB. The centroid of each detected ball was then converted from image coordinates to real world coordinates using the checkerboard surface which the camera was calibrated to. These coordinates were then adjusted for height using similar triangle analysis, turning our 2D camera image into an accurate prediction of the ball's real X, Y, and Z coordinates in the robot's base frame.

Using these corrected coordinates, the robot was able to then generate a trajectory to move above the ball, and then descend, grab, and drop it into sorted bins based on the color of the ball. The cubic trajectory generator caused smooth motions with minimal jerk, and no sudden changes in velocity. Directing the robot to corrected coordinates ensured that the robot never collided with the ground.

The implementation of forward and inverse kinematics made it possible for the robot to move to the balls reliably. The inverse kinematics constantly produced valid joint configurations given coordinate of the balls. While the forward kinematics gave us accurate readings of the joint angles themselves. We also implemented differential Kinematics, which successfully detected singularities, and disallowed the robot to attempt to go into impossible joint configurations.

Overall, the system was able to repeatedly detect and sort balls that were placed on the grid that was placed within the camera's field of view. The state machine made it so that the robot could continually cycle between sorting and searching. When minor position errors or lighting changes occurred, the image processing pipeline was able to detect and pick up balls despite the errors.

IV. DISCUSSION

A. Kinematic Discussion

Our kinematic approach worked well and enabled our robot to complete the necessary tasks for the project deliverables. While our *Gamma* value must remain constant throughout the robot's motion, we maintained the functionality of being able to choose the *Gamma* before running the robot. This means that while we cannot adjust the *Gamma* of the end effector wrist mid ball placement, we are able to change and update it between placements if needed to suit our needs, and our kinematic algorithms will handle the decoupling as needed. While geometrically deriving the kinematics may have been more time consuming as an approach, it allowed for a deeper understanding of our system and provided us with better insight as to what was physically happening with our robot.

B. Trajectory Discussion

The trajectory generator worked great on command points. We used our cubic trajectory to command the robot to move linearly in task space. We would choose points to go above the board so as not to have the arm fall into the board itself.

C. Computer Vision Discussion

As previously mentioned in the section pertaining to the Image Processing Pipeline, our implementation of Computer Vision was one of the greatest successes of our arm and its application. The utilization of multiple color masks served to be the most effective source of error reduction throughout the process of designing the logic, as isolating the colors, which was enabled by their comparative vibrancy to their surroundings, made it so then isolating the individual colors was a process unhindered by various surroundings. This isolation was discovered when adjusting the masks in the Color Calibrate MATLAB app, and was a direct response to our previous attempts having significant noise following color masking, as the colors can be difficult to distinguish, and adjustments of the ball colors were limited by the color profiles of the checkerboard and background. One hiccup

which was discovered was the recalculation of the static mask for each image taken. This of course goes against its static nature, but also made it so the checkerboard pattern would be broken by any ball on the surface, and thus the mask would cut off portions of the board. Due to the structure of our state machine, this would just lead to some balls on the board being cut out of the image, to be seen by another capture taken after the removal of some other balls. However, this was a minor problem solved by applying the mask to each image, but only calculating the mask at the time of the Image Processor's initialization, allowing any and all balls on the board to be detected with just one capture, and then collected in one passthrough of our Image Processing Pipeline and Robot control system.

D. Workflow & System Architecture Discussion & Justification

Throughout the course, we have utilized GitHub's desktop features to create and manage issues. These issues can be thought of as simplified tasks for group members and can range from actual code segments to segments of written work. This system has kept us organized and given us a roadmap for each lab thus far, allowing for organized and well distributed management of workload between members. This system not only helps us keep track of each members contributions (although not perfectly when team members use each other's devices to complete work) but also gives us a quick overview of our progress and how many tasks remain before the completion of the lab, reducing stress. This is definitely a system which we will continue to use in the future.

As for system architecture, we followed all lab guidelines in producing object-oriented code. This means independent files and objects control certain sets of functions that pertain only to a specific pipeline. For example, the robot class is an object that manages all aspects of robot motion; however, it calls upon other objects (trajectory and kinematic generators, computer vision, etc.) to maintain well managed implementation. Object oriented code allowed for us to expand upon each lab cleanly, as we did not have to redefine or create new versions of previously made functions, but rather called upon objects written between labs to implement new functionality and build upon previous goals. In addition to allowing for the building of larger projects from smaller work sessions, object-oriented code eases the stress of troubleshooting by isolating information pipelines between tasks. In other words, modifications to one object or method within an object should not interfere with already existing and functioning objects. Our group made use of the Open Closed Principle (OCP) to maintain stability between code expansions by adding new behavior without existing tested and working behavior.

V. CONCLUSION

Throughout this term, we have been given the opportunity to learn and implement a variety of topics and methods within the field of Robotic kinematics. Beginning this term, we had little knowledge of MATLAB, Linux, GitHub collaborative workflow, or reference frame translations for kinematic applications. To begin this term, we developed skills in installing Linux software, utilizing MATLAB graphing features, and setting up a collaborative Git environment. Moving forward, we built skills in geometric analysis of drawings to derive usable information about translation in space. We then applied that information to kinematic methods like the DH table and Jacobian to derive Forward and Inverse Kinematics for our robot. Using this information, we were able to implement joint space to task space conversions in MATLAB and begin to graph robot motion based on commands in task space. Over time, we built upon our previous code using Object Oriented methods, and week by week implemented new functionality, including trajectory planning for smooth robot motion as well as computer vision in the final lab for environmental sensing. Our final challenge of working with different reference frames came when we were asked to utilize computer vision to manipulate objects, using a camera mounted at a fixed position in space away from our robot and base reference frames. To solve this issue, we applied our knowledge from Lab 1 and built a reference frame for our image processing pipeline as well using real world measurements. In software, we also made use of computer vision features to normalize our fisheye lens, apply a vision mask to only intake relevant pixel data, and implemented an algorithm to compensate for our cameras lack of depth perception and inaccurate projection of point to world coordinates. Ultimately, this experience strengthened all our abilities as robotics engineers to work collaboratively on a project and utilize an object-oriented structure to command a robot with multiple frames of 3D reference space.

APPENDIX

[Click Here for GitHub Release](#) | [Click Here for Video](#)

TABLE OF CONTRIBUTIONS

CONTRIBUTIONS TABLE	PLANNING	MATHEMATICAL DERIVATION	MATLAB CODE	EXPERIMENTATION	ANALYSIS	DOCUMENTATION
F. SCHRODER	25	25	25	25	25	25
B. LAVOIE	25	25	25	25	25	25
S. DHINGRA	25	25	25	25	25	25
M. SACCA	25	25	25	25	25	25

a. Team Contributions Table, with percentage of each members contribution to a relevant task.